

// Day 19 of 30

COMPARISON + REAL CODE

Pessimistic vs Optimistic Locking

→ Swipe through all 6 slides



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

1/8

The Problem

Two users update the same row simultaneously

User A reads balance: 1000

User B reads balance: 1000

User A: $\text{balance} - 500 = 500 \rightarrow \text{save}$

User B: $\text{balance} - 300 = 700 \rightarrow \text{save}$

Final balance: 700

Expected: 200

→ Lost Update Problem

→ User A's deduction vanished

💡 Without locking, concurrent writes corrupt data



Pessimistic Locking

Lock the row. Nobody touches it.

```
@Query("SELECT w FROM Wallet w  
      WHERE w.id = :id")  
@Lock(LockModeType.PESSIMISTIC_WRITE)  
Wallet findByIdForUpdate(Long id);
```

- Database locks the row
- Other transactions WAIT
- Lock released after commit/rollback

💡 Use when: high contention (payments, inventory)



Optimistic Locking

No lock. Check version on save.

```
@Entity
public class Product {
    @Version
    private int version;
}
```

- Read: version = 1
- Update: SET version = 2
WHERE id = 5 AND version = 1
- If someone else changed it:
 - version $\neq$ 1 → UPDATE fails
 - OptimisticLockException thrown

💡 Use when: low contention (profiles, settings)



Side-by-Side Comparison

When to use what

	Pessimistic	Optimistic
Locks DB row:	Yes	No
Performance:	Slower	Faster
Conflict:	Waits	Retries
Best for:	Payments	Profiles
	Inventory	Settings
	Bank balance	User prefs
Risk:	Deadlocks	Stale data

💡 High contention → Pessimistic. Low → Optimistic.



Deadlock Prevention

Pessimistic locking's biggest risk

Deadlock:

Thread A locks Wallet 1, waits for 2
Thread B locks Wallet 2, waits for 1
→ Both stuck forever

Prevention:

Always lock in SAME ORDER
(lower ID first)

```
if (fromId < toId) {  
    lock(fromId); lock(toId);  
} else {  
    lock(toId); lock(fromId);  
}
```

💡 Ordered locking = zero deadlocks. Always.



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Real-World Usage

What I built in PayFlow

PayFlow digital wallet system:

- Pessimistic locking for transfers
- Ordered wallet locking (lower ID first)
- Idempotency keys (no double payments)
- Result: zero deadlocks, zero duplicates

github.com/Shubh2-0/PayFlow

💡 Build it. Deploy it. Talk about it in interviews.



Shubham Bhati

linkedin.com/in/bhatishubham

done = true

Follow for more!

Like · Comment · Repost

Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)